PROGRAMMING PROJECT 2

Due: 30/05/2022 23:00

In this project, you are required to extend the MIPS single-cycle implementation by implementing additional instructions. You will use ModelSim simulator [1] to develop and test your code. You are required to implement your assigned **6 instructions** (selected from following 29 instructions). *Note that your set of **6 instructions** will be emailed to you or your group member.*

R-format (11): brn, brz, balrn, balrz, sll, srl, sllv, srlv, jmadd, jmor, jmsub

I-format (13):bmz, bmn, bmv, balmz, balmn, balmv, jm, jalm, jrs, jrsal, jpc, jsp, jspal

J-format (5): bn, bz, baln, balz, jal

You must design the revised single-cycle datapath and revised control units which make a processor that executes your instructions as well as the instructions implemented already in the design. After designing the new enhanced processor, you will implement it in Verilog HDL.

Specifications of New Instructions

	Instr	Type	Code	Syntax	Meaning
1.	bmz	I-type	opcode=20	bmz imm16($rs)	if Status [Z] = 1, branches to address found in memory
2.	bmn	I-type	opcode=21	bmn imm16($rs)	if Status [N] = 1, branches to address found in memory
3.	bmv	I-type	opcode=22	bmv imm16($rs)	if Status [V] = 1, branches to address found in memory
4.	balmz	I-type	opcode=23	balmz $rt, imm16($rs)	if Status [Z] = 1, branches to address found in memory, link address is stored in $rt
5.	balmn	I-type	opcode=24	balmn $rt, imm16($rs)	if Status [N] = 1, branches to address found in memory, link address is stored in $rt
6.	balmv	I-type	opcode=25	balmv $rt, imm16($rs)	if Status [V] = 1, branches to address found in memory, link address is stored in $rt
7.	brz	R-type	funct=20	brz $rs	if Status [Z] = 1, branches to address found in register $rs.
8.	brn	R-type	funct=21	brn $rs	if Status [N] = 1, branches to address found in register $rs.
9.	balrz	R-type	funct=22	balrz $rs, $rd	if Status [Z] = 1, branches to address found in register $rs link address is stored in $rd (which defaults to 31)
10.	balrn	R-type	funct=23	balrn $rs, $rd	if Status [N] = 1, branches to address found in register $rs link address is stored in $rd (which defaults to 31)
11.	jmadd	R-type	funct=32	jmadd $rs,$rt	jumps to address found in memory [$rs+$rt], link address is stored in $31

12. jmor	R-type funct=37	jmor $rs,$rt	jumps to address found in memory [$rs\|$rt], link address is stored in $31
13. jmsub	R-type funct=34	jmsub $rs,$rt	jumps to address found in memory [$rs-$rt], link address is stored in $31
14. bz	J-type opcode=24	bz Target	if Status [Z] = 1, branches to pseudo-direct address (formed as j does)
15. bn	J-type opcode=25	bn Target	if Status [N] = 1, branches to pseudo-direct address (formed as j does)
16. balz	J-type opcode=26	balz Target	if Status [Z] = 1, branches to pseudo-direct address (formed as jal does), link address is stored in register 31
17. baln	J-type opcode=27	baln Target	if Status [N] = 1, branches to pseudo-direct address (formed as jal does), link address is stored in register 31
18. jm	I-type opcode=18	jm imm16($rs)	jumps to address found in memory (indirect jump)
19. jalm	I-type opcode=19	jalm $rt, imm16($rs)	jumps to address found in memory (indirect jump), link address is stored in $rt (which defaults to 31)
20. jrs	I-type opcode=18	jrs $rs	jumps to address found in memory where the memory address is written in register $rs.
21. jrsal	I-type opcode=19	jrsal $rs	jumps to address found in memory where the memory address is written in register $rs and link address is stored in memory (DataMemory[$rs]).
22. sll	R-type func=0	sll $rd, $rt, shamt	shift register $rt to left by shift amount (shamt) and store the result in register $rd.
23. srl	R-type func=2	srl $rd, $rt, shamt	shift register $rt to right by shift amount (shamt) and store the result in register $rd.
24. sllv	R-type func=4	sllv $rd, $rt, $rs	shift register $rt to left by the value in register $rs, and store the result in register $rd.
25. srlv	R-type func=6	srlv $rd, $rt, $rs	shift register $rt to right by the value in register $rs, and store the result in register $rd.
26. jpc	I-type opcode=30	jpc $rt, Target	jumps to PC-relative address (formed as beq and bne do) link address is stored in $rt (which defaults to 31)
27. jsp	I-type opcode=18	jsp	jumps to address found in memory where the memory address is written in register 29 ($sp).
28. jspal	I-type opcode=19	jspal	jumps to address found in memory where the memory address is written in register 29 ($sp) and link address is stored in memory (DataMemory[Register[29]]).
29. jal	J-type opcode=3	jal Target	jump to pseudo-direct address (formed as j does), link address is stored in register 31.

Status Register

Some of the conditional branches test the Z and N bits in the Status register. So the MIPS datapath will need to have a Status register, with the following 3 bits: Z (if the ALU result is zero) , N (if the ALU result is negative) or V (if the ALU result causes overflow). The Status register will be loaded with the ALU results each clock cycle.

Requirements

- You will implement only 6 instructions. Therefore, you have to send a request email to lokman.altin@gmail.com with introducing your group members (3 or 4) as soon as possible.
- The deadline for the instruction request is **16.05.2022 (Monday) 05:00pm**. Please do not send me an email later than that date.
- Your design should support all your 6 instructions without corrupting other MIPS instructions.
- You can print out single-cycle datapath and start your design on a paper. Then you can continue with the Verilog implementation.
- A demo session will be scheduled in the following week of the submission. I request you to bring these design papers within you in demo sessions.
- **You have to prepare a simulation of your instructions for the demo (including *register file*, *data memory* and *instruction memory*)!**
- You have to do your projects in Verilog with using ModelSim simulator.
- You are allowed to use only the source codes in the attached. You cannot take any other implementations.
- You **will not** submit 6 different implementations. There should be only a single implementation supporting all 6 instructions that are assigned to your group.
- You are required to submit a minimum 2-pages report explaining implementation details of your project and commented code (via Canvas). Your report should include example runs for each instruction. Your implementation detail should be provided in the source code comment. All the files should be submitted as a single zip file. You should use your surnames as the name of the file: *surname1_surname2_surname3_surname4_project2.zip*

General Policies

- *You are allowed to work in groups of 3 or 4 members. Group members may get different grades.*
- *It is NOT acceptable to copy (or start your) solutions from Web. In case of any forms of cheating or copying, the penalties will be severe.* ***Both Giver and Receiver are equally culpable and suffer equal penalties.***
- There will be demo session for this assignment. If you cannot answer the questions about your project details in the demo section, even if you have done all the parts completely, you will not get points!
- ***No late submission will be accepted!***

[1] https://www.intel.com/content/www/us/en/software/programmable/quartus-prime/model-sim.html